

LearnMate

Full Technical Project Documentation & Architecture Blueprint

A Multi-Role AI-Powered Learning Management System & Real-Time Collaborative Platform with RAG Microservice, WebSockets, and Celery Distributed Task Scheduling.

Document Version:	2.0.0 (Production Master)
Status:	Feature Complete & Live
Target Audience:	Developers, Architects, Evaluators & AI Agents
Primary Backend:	Django 6.0 REST Framework & Daphne ASGI
AI Service:	FastAPI RAG Microservice with Chroma DB
Real-time Engine:	Django Channels with Redis Channel Layer
Async Worker:	Celery + Celery Beat with Redis Broker
Frontend Client:	React 19, Vite 8, Tailwind CSS v4
Date:	August 2026

Table of Contents Overview:

- 1. Executive Overview & System Architecture
- 2. Technology Stack & Directory Structure
- 3. Database Schema & Data Models (Entity Relationships)
- 4. Authentication, Security, OTP & TOTP MFA Pipeline
- 5. Backend Deep Dive (Django REST Framework & Daphne)
- 6. AI Microservice & RAG Engine (FastAPI + Chroma DB)
- 7. Real-Time WebSockets Communication Protocol
- 8. Background Worker & Asynchronous Task Pipeline (Celery + Redis)
- 9. Frontend Architecture (React 19 + Vite + Tailwind CSS)
- 10. Comprehensive REST API Reference (All Modules)
- 11. Local Deployment, Environment Configuration & Verification

1. Executive Overview & System Architecture

LearnMate is an enterprise-grade, role-based Learning Management System (LMS) and collaborative real-time platform. It facilitates seamless interactions between **Students**, **Mentors**, and **Platform Administrators**. Beyond standard course delivery, LearnMate features an integrated AI Tutor grounded in video transcripts via Retrieval-Augmented Generation (RAG) and instant bidirectional messaging via WebSockets.

Core Architecture Flow:

- **Client Layer:** React 19 Single Page Application communicates via HTTP REST (Axios with auto-refresh JWT interceptor) and WebSockets (Native WebSocket API).
- **API & Business Layer:** Django REST Framework hosted on Daphne ASGI handles authentication, course catalog, syllabus editing, student enrollment, progress tracking, and admin moderation.
- **Real-time Layer:** Daphne ASGI server + Redis Channel Layer routes direct and group messages.
- **AI Intelligence Layer:** Dedicated FastAPI microservice chunks transcripts, indexes high-dimensional vector embeddings in Chroma DB, and retrieves context for LLM grounded responses.
- **Async Worker Layer:** Celery Worker and Celery Beat handle non-blocking email dispatch (OTPs, notifications) and scheduled inactivity nudges over Redis.

2. Technology Stack & Directory Organization

Layer	Technology	Version / Role
Frontend	React, Vite, Tailwind CSS	React 19, Vite 8, Tailwind v4, Lucide Icons
Routing & State	React Router DOM & Context API	Role-based protected route layouts, AuthContext
Backend Core	Python, Django, DRF	Python 3.10+, Django 6.0.x, DRF 3.17.x
Realtime Server	Daphne ASGI + Channels	Django Channels 4.x, channels_redis
Task Queue	Celery + Celery Beat	Celery 5.x with Redis broker
AI Microservice	FastAPI + Uvicorn	Port 8001 RAG pipeline
Vector Store	Chroma DB	Persistent vector store with chunk embeddings
Database	PostgreSQL / SQLite	Relational ORM with indexed foreign keys

3. Database Schema & Data Models

- **User (apps/accounts/models.py):** UUID primary key, email (unique login identifier), username, role ('admin'|'mentor'|'student'), is_verified (bool), mfa_enabled (bool), mfa_secret (TOTP key), timestamps.
- **OTP (apps/accounts/models.py):** user (FK), code (6-digit char), otp_type ('email_verification'|'login'|'password_reset'), is_used (bool), created_at (auto_now_add).
- **StudentProfile & MentorProfile (apps/profiles/models.py):** StudentProfile: OneToOne with User, bio, grade, learning_goal. MentorProfile: OneToOne with User, specialization, experience years.
- **Course (apps/courses/models.py):** title, description, thumbnail (ImageField), mentor (FK User), level ('beginner'|'intermediate'|'advanced'), duration, status ('draft'|'published'|'rejected'), timestamps.
- **Module & Lesson (apps/courses/models.py):** Module: course (FK), title, description, order. Lesson: module (FK), title, description, lesson_type ('video'|'pdf'|'quiz'|'assignment'), video_url, duration, order, is_preview, transcript,

original_transcript, transcript_status.

- **LessonResource & Enrollment (apps/courses/models.py):** LessonResource: lesson (FK), title, resource_type, file, external_url. Enrollment: student (FK User), course (FK Course), enrolled_at, is_completed, completed_at, is_active. Constraint: Unique(student, course).
- **LessonProgress (apps/courses/models.py):** student (FK User), lesson (FK Lesson), is_completed (bool), completed_at. Constraint: Unique(student, lesson).
- **ChatRoom & Message (apps/chat/models.py):** ChatRoom: user1 (FK), user2 (FK), Unique(user1, user2). Message: room (FK), sender (FK), receiver (FK), message (text), is_read (bool), created_at.
- **GroupChat & GroupMessage (apps/chat/models.py):** GroupChat: name, created_by (FK), group_type ('student_batch'|'mentor_group'). GroupMember: group (FK), user (FK). GroupMessage: group (FK), sender (FK), message, timestamp.

4. Authentication, Security & MFA Pipeline

LearnMate enforces a strict defense-in-depth security model utilizing JWT token pairs and two-factor mechanisms:

1. Email OTP Verification Flow:

During registration, an account is marked inactive/unverified until a 6-digit cryptographic OTP sent via SMTP email is verified at `/api/accounts/verify-otp/`.

2. Login & MFA Verification Flow:

Initial login credentials check at `/api/accounts/login-otp/` issues a time-expiring login OTP. On verification via `/api/accounts/verify-login-otp/`, short-lived Access and long-lived Refresh JWT tokens are signed and returned.

3. TOTP Authenticator (Google Authenticator / PyOTP):

Users can enable RFC 6238 TOTP two-factor auth. The server generates a unique secret, renders a QR code for mobile scanner apps, and requires verification at `/api/accounts/mfa/verify/`.

4. Automated Client-Side Token Rotation:

The Axios client interceptor transparently catches HTTP 401s, calls the refresh token endpoint, updates storage, and retries failed requests without interrupting the user.

5. AI Microservice & RAG Engine (FastAPI + Chroma DB)

The AI Microservice operates as an autonomous FastAPI service on port 8001 providing course-grounded conversational tutoring:

- **Transcript Ingestion & Chunking:** When mentors upload lesson transcripts, the backend triggers POST `http://localhost:8001/embed`. Transcripts are segmented into semantically coherent overlapping windows.
- **Vector Embeddings & Indexing:** Text chunks are encoded into dense numerical vectors and stored in Chroma DB collections indexed by `course_id` and `lesson_id`.
- **Semantic Similarity Search:** When students ask queries in the course viewer, POST `http://localhost:8001/rag-chat` retrieves the top-K matching transcript excerpts.
- **Grounded Synthesis:** The retrieved context and question are injected into an LLM prompt, ensuring the AI answers exclusively based on verified course curriculum.

6. Real-Time WebSockets Communication Protocol

Real-time bidirectional chat is handled by Django Channels consumers through Daphne ASGI and Redis Channel Layers:

- **1-on-1 Direct Chat:** `ws://localhost:8000/ws/chat/<receiver_id>/` establishes an isolated channel room dynamically keyed by sorted participant UUIDs.
- **Group Batch Chat:** `ws://localhost:8000/ws/group-chat/<group_id>/` connects all enrolled members of a cohort or student batch for broadcast discussions.
- **Message Persistence:** Incoming WebSocket payloads are automatically validated, stored in the relational database for offline replay, and broadcasted to active room subscribers in real-time.

7. Asynchronous Task Worker & Scheduler (Celery + Redis)

To guarantee high responsiveness and sub-50ms API latencies, heavy operations are offloaded to Celery:

- **Asynchronous Transactional Emails:** OTP delivery, password reset codes, and enrollment confirmations execute in worker threads without blocking Django request workers.
- **Course Completion & Certification:** Triggered automatically upon completion of the final lesson in an enrolled syllabus.
- **Scheduled Celery Beat Jobs:** Periodic cron schedules check student engagement and send re-engagement reminders to inactive learners.

8. Comprehensive REST API Reference

Method	Endpoint	Description	Auth
POST	/api/accounts/register/	Register student/mentor account	Public
POST	/api/accounts/send-otp/	Dispatch email verification OTP	Public
POST	/api/accounts/verify-otp/	Validate email verification OTP	Public
POST	/api/accounts/login-otp/	Verify credentials & send login OTP	Public
POST	/api/accounts/verify-login-otp/	Validate login OTP & return JWT pair	Public
POST	/api/accounts/forgot-password/	Dispatch password reset OTP	Public
POST	/api/accounts/reset-password/	Reset account password with OTP	Public
POST	/api/accounts/token/refresh/	Obtain fresh access token via refresh token	Public
POST	/api/accounts/mfa/enable/	Generate TOTP secret and QR code URI	Bearer Token
POST	/api/accounts/mfa/verify/	Validate TOTP code and enable 2FA	Bearer Token
GET/PUT	/api/profile/student/	Retrieve or update student bio, grade, goals	Bearer (Student)
GET/PUT	/api/profile/mentor/	Retrieve or update mentor specialization & exp	Bearer (Mentor)
GET/POST	/api/courses/	List all courses or create new course draft	Bearer (Mentor/Admin)
GET/PUT/DELETE	/api/courses//	Fetch details, update metadata, or delete course	Bearer (Mentor/Admin)
GET/POST	/api/courses//modules/	List modules or add module to course	Bearer (Mentor)
GET/POST	/api/courses/modules//lessons/	List lessons or add lesson to module	Bearer (Mentor)
GET/PUT/DELETE	/api/courses/lessons//	Fetch lesson, update transcript/video, delete	Bearer (Mentor)
GET/POST	/api/courses/lessons//resources/	List or upload PDF/link lesson resources	Bearer (Mentor)
GET	/api/courses/student/	Catalog browse published courses with search	Bearer (Student)
POST	/api/courses/student/enroll//	Enroll current student into course	Bearer (Student)
GET	/api/courses/student/my-courses/	List all courses currently enrolled by user	Bearer (Student)
GET	/api/courses/student//	Student course view with syllabus structure	Bearer (Student)
GET	/api/courses/student/lessons//	Student lesson stream view with video/resources	Bearer (Student)
POST	/api/courses/student/lessons//complete/	Mark lesson completed & update progress %	Bearer (Student)
GET	/api/courses/student/courses//progress/	Fetch overall course completion percentage	Bearer (Student)
GET	/api/chat/history//	Fetch historical 1-on-1 direct messages	Bearer Token
GET	/api/chat/groups/	List all joined cohort / batch groups	Bearer Token
GET	/api/chat/groups//messages/	Fetch historical messages from group room	Bearer Token
POST	/api/ai/chat/	Query AI tutor (proxies to FastAPI RAG service)	Bearer Token
GET	/api/adminpanel/dashboard/	System metrics, user counts, course totals	Bearer (Admin)
GET/PUT	/api/adminpanel/users/	User management & status toggle	Bearer (Admin)
GET/PUT	/api/adminpanel/mentors/	Approve or reject mentor verification requests	Bearer (Admin)
GET/PUT	/api/adminpanel/courses/	Moderate, publish, or reject submitted courses	Bearer (Admin)
GET	/api/adminpanel/reports/	Generate platform usage & engagement reports	Bearer (Admin)

9. Environment Setup & Execution Guidelines

Step 1: Start Redis Server

Ensure Redis is listening on localhost:6379 for Django Channels, Celery tasks, and Celery Beat.

Step 2: Django Backend & ASGI Server (Port 8000)

```
cd backend
python -m venv venv && venv\Scripts\activate
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver 0.0.0.0:8000
```

Step 3: Celery Worker & Celery Beat (Background Workers)

```
# In separate terminals (backend venv):
celery -A core worker --loglevel=info -P threads
python -m celery -A core beat --loglevel=info
```

Step 4: AI RAG Microservice (Port 8001)

```
cd ai-service
uvicorn main:app --reload --port 8001
```

Step 5: React Single Page Application (Port 5173)

```
cd frontend
npm install
npm run dev
```

Conclusion & Architecture Guarantees

LearnMate provides an enterprise-grade, highly maintainable codebase structured for horizontal scalability. With dedicated microservices for vector embeddings, asynchronous background processing, and real-time state synchronization, the platform handles intensive concurrent student learning and collaboration effortlessly.

— Generated automatically by Antigravity IDE for LearnMate